

Denotational Semantics of a Relational λ -Calculus

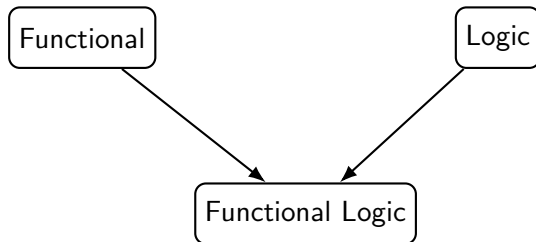
Mariana Milicich

(based on joint work with Pablo Barenbaum and Federico Lochbaum)

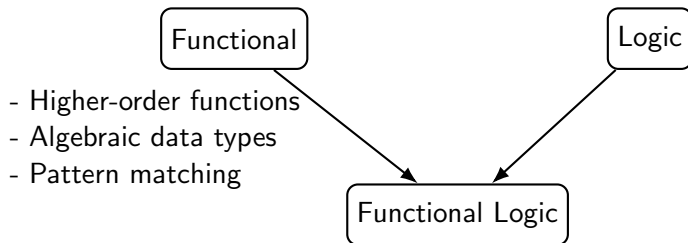
University of Tartu

June 4, 2026
CS Theory Seminar

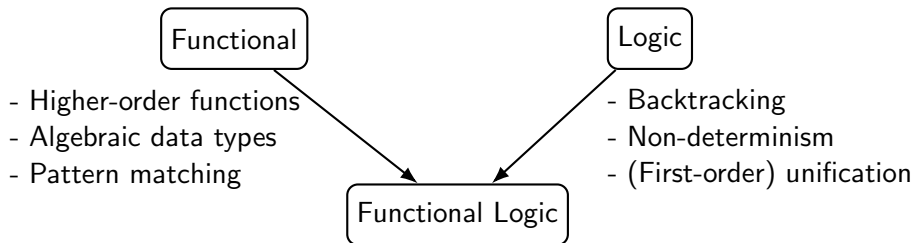
Functional Logic Programming



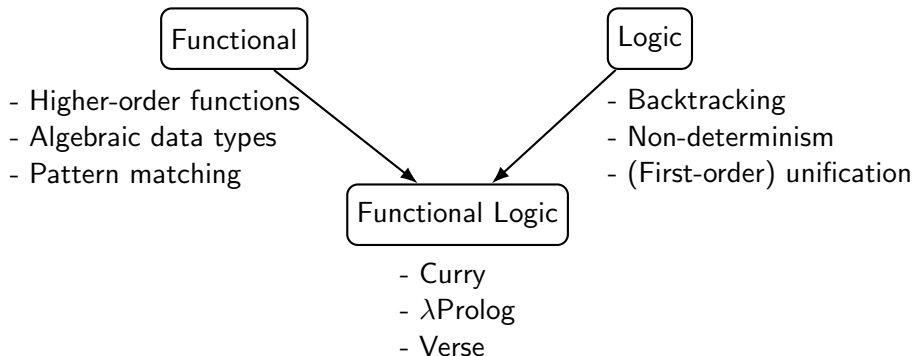
Functional Logic Programming



Functional Logic Programming



Functional Logic Programming



Contributions

- A type system for the λ^U -calculus (Barenbaum and Lochbaum, 2019).
 - Type Preservation.
- A denotational semantics for the typed fragment of λ^U .
 - (Weak) Soundness.
 - Compositionality.
 - ~~Completeness.~~

Contributions

- A type system for the λ^U -calculus (Barenbaum and Lochbaum, 2019).
 - Type Preservation.
- A denotational semantics for the typed fragment of λ^U .
 - (Weak) Soundness.
 - Compositionality.
 - ~~Completeness.~~

- One aim of the talk is to bring back this topic.
- By no means I was nor am an expert in denotational semantics. Any suggestions and/or corrections are more than welcome!

The λ^U -Calculus (Barenbaum and Lochbaum, 2019)

Terms, Programs, and Values

$$M, N ::= x \mid \mathbf{c} \mid \lambda x. P \mid \lambda^\ell x. P \mid M N \mid M \doteq N \mid M ; N \mid \nu x. M$$

$$P ::= \text{fail} \mid M \oplus P$$

$$V ::= x \mid \lambda^\ell x. P \mid \mathbf{c} V_1 \dots V_n \quad (n \in \mathbb{N})$$

The λ^U -Calculus (Barenbaum and Lochbaum, 2019)

Terms, Programs, and Values

$$M, N ::= x \mid \mathbf{c} \mid \lambda x. P \mid \lambda^\ell x. P \mid M N \mid M \dot{=} N \mid M ; N \mid \nu x. M$$

$$P ::= \text{fail} \mid M \oplus P$$

$$V ::= x \mid \lambda^\ell x. P \mid \mathbf{c} V_1 \dots V_n \quad (n \in \mathbb{N})$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

$$\{\lambda^\ell x. P \doteq \lambda^\ell x. P\} \uplus G \Rightarrow G$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

$$\{\lambda^\ell x. P \doteq \lambda^\ell x. P\} \uplus G \Rightarrow G$$

$$\{\mathbf{c} \ V_1 \ \dots \ V_n \doteq \mathbf{c} \ W_1 \ \dots \ W_n\} \uplus G \Rightarrow \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \uplus G$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

$$\{\lambda^{\ell}x.P \doteq \lambda^{\ell}x.P\} \uplus G \Rightarrow G$$

$$\{\mathbf{c} \ V_1 \ \dots \ V_n \doteq \mathbf{c} \ W_1 \ \dots \ W_n\} \uplus G \Rightarrow \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \uplus G$$

$$\{V \doteq W\} \uplus G \Rightarrow \text{FAIL} \quad \text{if } V \text{ and } W \text{ clash}$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

$$\{\lambda^\ell x. P \doteq \lambda^\ell x. P\} \uplus G \Rightarrow G$$

$$\{\mathbf{c} \ V_1 \ \dots \ V_n \doteq \mathbf{c} \ W_1 \ \dots \ W_n\} \uplus G \Rightarrow \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \uplus G$$

$$\{V \doteq W\} \uplus G \Rightarrow \text{FAIL} \quad \text{if } V \text{ and } W \text{ clash}$$

$$\{x \doteq V\} \uplus G \Rightarrow \{x \doteq V\} \uplus G\{x := V\}$$

Unification Algorithm

Unification Problem

A *unification problem* is a finite set G of the form:

$$G = \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \quad (n \in \mathbb{N})$$

Variant of the Martelli and Montanari Unification Algorithm

$$\{x \doteq x\} \uplus G \Rightarrow G$$

$$\{V \doteq x\} \uplus G \Rightarrow \{x \doteq V\} \uplus G \quad \text{if } V \text{ is not a variable}$$

$$\{\lambda^{\ell}x.P \doteq \lambda^{\ell}x.P\} \uplus G \Rightarrow G$$

$$\{\mathbf{c} \ V_1 \ \dots \ V_n \doteq \mathbf{c} \ W_1 \ \dots \ W_n\} \uplus G \Rightarrow \{V_1 \doteq W_1, \dots, V_n \doteq W_n\} \uplus G$$

$$\{V \doteq W\} \uplus G \Rightarrow \text{FAIL} \quad \text{if } V \text{ and } W \text{ clash}$$

$$\{x \doteq V\} \uplus G \Rightarrow \{x \doteq V\} \uplus G\{x := V\}$$

$$\{x \doteq V\} \uplus G \Rightarrow \text{FAIL} \quad \text{if } x \neq V \text{ and } x \in \text{fv}(V)$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^{\ell} x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{x} \doteq \mathbf{y}, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{\underline{x \doteq (\lambda^\ell x. \mathbf{c})}, \mathbf{x} \doteq \mathbf{y}, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), x \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), (\lambda^\ell x. \mathbf{c}) \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), x \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), (\lambda^\ell x. \mathbf{c}) \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), x \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), (\lambda^\ell x.c) \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), y \doteq (\lambda^\ell x.c), (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\{(\lambda^\ell x. \mathbf{c}) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), \mathbf{d} \ x \ (\lambda^{\ell'} z. z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z. z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), x \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), (\lambda^\ell x. \mathbf{c}) \doteq y, (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x. \mathbf{c}), y \doteq (\lambda^\ell x. \mathbf{c}), (\lambda^{\ell'} z. z) \doteq \lambda^{\ell'} z. z\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.c) \doteq x, \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), \mathbf{d} \ x \ (\lambda^{\ell'} z.z) \doteq \mathbf{d} \ y \ (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), x \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), (\lambda^\ell x.c) \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), y \doteq (\lambda^\ell x.c), (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.c), y \doteq \lambda^\ell x.c\}$$

Example

Let us apply the unification algorithm to the unification problem:

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), x \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), (\lambda^\ell x.\mathbf{c}) \doteq y, (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), y \doteq (\lambda^\ell x.\mathbf{c}), (\lambda^{\ell'} z.z) \doteq \lambda^{\ell'} z.z\}$$

$$\Rightarrow \{x \doteq (\lambda^\ell x.\mathbf{c}), y \doteq \lambda^\ell x.\mathbf{c}\}$$

$$\text{mgu}(\{(\lambda^\ell x.\mathbf{c}) \doteq x, \mathbf{d} x (\lambda^{\ell'} z.z) \doteq \mathbf{d} y (\lambda^{\ell'} z.z)\}) = \{x \mapsto (\lambda^\ell x.\mathbf{c}), y \mapsto \lambda^\ell x.\mathbf{c}\}$$

Operational Semantics of λ^U

$$W ::= \square \mid W M \mid M W \mid W \dot{=} M \mid M \dot{=} W \mid W ; M \mid M ; W$$

$W\langle\lambda x.P\rangle$	$\rightarrow$	$W\langle\lambda^\ell x.P\rangle$	where ℓ is fresh
$W\langle(\lambda^\ell x.P) V\rangle$	$\rightarrow$	$W\langle P\{x := V\}\rangle$	
$W\langle V ; M\rangle$	$\rightarrow$	$W\langle M\rangle$	
$W\langle\nu x.M\rangle$	$\rightarrow$	$W\langle M\{x := y\}\rangle$	where y is fresh
$W\langle V \dot{=} W\rangle$	$\rightarrow$	$W\langle\mathbf{ok}\rangle^\sigma$	if there exists $\sigma := \text{mgu}(\{V \dot{=} W\})$
$W\langle V \dot{=} W\rangle$	$\rightarrow$	$W\langle\mathbf{fail}\rangle$	if $\text{mgu}(\{V \dot{=} W\})$ fails

Operational Semantics of λ^U

$$W ::= \square \mid W M \mid M W \mid W \dot{=} M \mid M \dot{=} W \mid W ; M \mid M ; W$$

$$\begin{aligned} W\langle\lambda x.P\rangle &\rightarrow W\langle\lambda^\ell x.P\rangle && \text{where } \ell \text{ is fresh} \\ W\langle(\lambda^\ell x.P) V\rangle &\rightarrow W\langle P\{x := V\}\rangle \\ W\langle V ; M\rangle &\rightarrow W\langle M\rangle \\ W\langle\nu x.M\rangle &\rightarrow W\langle M\{x := y\}\rangle && \text{where } y \text{ is fresh} \\ W\langle V \dot{=} W\rangle &\rightarrow W\langle\mathbf{ok}\rangle^\sigma && \text{if there exists } \sigma := \text{mgu}(\{V \dot{=} W\}) \\ W\langle V \dot{=} W\rangle &\rightarrow W\langle\mathbf{fail}\rangle && \text{if } \text{mgu}(\{V \dot{=} W\}) \text{ fails} \end{aligned}$$

$$\frac{P \rightarrow M_1 \oplus \dots \oplus M_n \quad (n \in \mathbb{N})}{Q \oplus P \oplus Q' \rightarrow Q \oplus M_1 \oplus \dots \oplus M_n \oplus Q'}$$

Example: pred 2

$$\underbrace{(\lambda^{\ell} x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} y); y))}_{\text{pred 2}} (\mathbf{s} (\mathbf{s} \mathbf{0}))$$

Example: pred 2

$$(\lambda^{\ell} x. ((x \doteq 0); \text{none}) \oplus \nu y. \text{just } ((x \doteq s\ y); y)) (s\ (s\ 0))$$

$$((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y. \text{just } ((s\ (s\ 0) \doteq s\ y); y)$$

Example: pred 2

$$(\lambda^{\ell}x.((x \doteq 0); \text{none}) \oplus \nu y.\text{just } ((x \doteq s\ y); y)) (s\ (s\ 0))$$

$$((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y.\text{just } ((s\ (s\ 0) \doteq s\ y); y)$$

Unification fails

Example: pred 2

$$\begin{array}{c} (\lambda^{\ell} x. ((x \doteq 0); \text{none}) \oplus \nu y. \text{just } ((x \doteq s\ y); y)) (s\ (s\ 0)) \\ \downarrow \\ ((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y. \text{just } ((s\ (s\ 0) \doteq s\ y); y) \\ \underbrace{\hspace{10em}}_{\text{Unification fails}} \quad \downarrow \\ \text{fail} \oplus \nu y. \text{just } ((s\ (s\ 0) \doteq s\ y); y) \end{array}$$

Example: pred 2

$$(\lambda^{\ell} x. ((x \dot{=} 0); \text{none}) \oplus \nu y. \text{just } ((x \dot{=} s\ y); y)) (s\ (s\ 0))$$

$$((s\ (s\ 0) \dot{=} 0); \text{none}) \oplus \nu y. \text{just } ((s\ (s\ 0) \dot{=} s\ y); y)$$

$$\nu y. \text{just } ((s\ (s\ 0) \dot{=} s\ y); y)$$

Example: pred 2

$$(\lambda^{\ell}x.((x \doteq 0); \text{none}) \oplus \nu y.\text{just } ((x \doteq s\ y); y)) (s\ (s\ 0))$$

$$((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y.\text{just } ((s\ (s\ 0) \doteq s\ y); y)$$

$$\nu y.\text{just } ((s\ (s\ 0) \doteq s\ y); y)$$

$$\text{just } ((s\ (s\ 0) \doteq s\ z); z)$$

Example: pred 2

$$(\lambda^{\ell} x. ((x \doteq 0); \text{none}) \oplus \nu y. \text{just } ((x \doteq s\ y); y)) (s\ (s\ 0))$$



$$((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y. \text{just } ((s\ (s\ 0) \doteq s\ y); y)$$



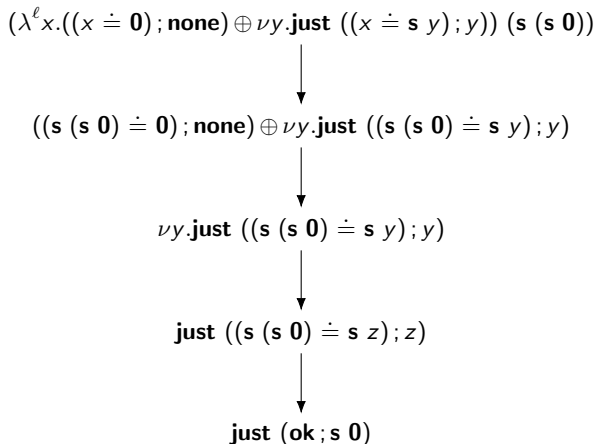
$$\nu y. \text{just } ((s\ (s\ 0) \doteq s\ y); y)$$



$$\text{just } ((s\ (s\ 0) \doteq s\ z); z)$$

There exists $\text{MGU}(\underbrace{\{s\ (s\ 0) \doteq s\ z\}}) = \{z \mapsto s\ 0\}$

Example: pred 2



Example: pred 2

$(\lambda^{\ell}x.((x \doteq 0); \text{none}) \oplus \nu y.\text{just } ((x \doteq s\ y); y)) (s\ (s\ 0))$



$((s\ (s\ 0) \doteq 0); \text{none}) \oplus \nu y.\text{just } ((s\ (s\ 0) \doteq s\ y); y)$



$\nu y.\text{just } ((s\ (s\ 0) \doteq s\ y); y)$



$\text{just } ((s\ (s\ 0) \doteq s\ z); z)$



$\text{just } (\text{ok}; s\ 0)$



$\text{just } (s\ 0)$

Simple Types for λ^U Terms

Grammar of types:

$$A, B ::= \alpha \mid A \rightarrow B$$

Grammar of typing contexts:

$$\Gamma ::= \cdot \mid \Gamma, x : A$$

Simple Types for λ^U Terms

Grammar of types:

$$A, B ::= \alpha \mid A \rightarrow B$$

Grammar of typing contexts:

$$\Gamma ::= \cdot \mid \Gamma, x : A$$

Note: the type system uses intrinsic typing.

$$x^A \mid \lambda x^A. P \mid \lambda^\ell x^A. P \mid \nu x^A. M \mid \text{fail}^A$$

Typing Rules

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x^A : A}$$

$$\overline{\Gamma \vdash \mathbf{c} : \mathcal{T}_c}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B}$$

$$\frac{\Gamma, x : A \vdash P : B}{\Gamma \vdash \lambda x^A. P : A \rightarrow B}$$

$$\frac{\Gamma, x : A \vdash P : B}{\Gamma \vdash \lambda^\ell x^A. P : A \rightarrow B}$$

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash M \doteq N : \mathcal{T}_{\text{ok}}}$$

$$\frac{\Gamma \vdash M : \mathcal{T}_{\text{ok}} \quad \Gamma \vdash N : A}{\Gamma \vdash M ; N : A}$$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \nu x^A. M : B}$$

$$\overline{\Gamma \vdash \text{fail}^A : A}$$

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash P : A}{\Gamma \vdash M \oplus P : A}$$

Example of a Typing Derivation

$$\frac{}{\cdot \vdash (\lambda^{\ell} x. ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}}$$

Example of a Typing Derivation

$$\frac{\vdots \quad \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash (\lambda^{\ell} x. ((x \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} y) ; y)) (\mathbf{s} (\mathbf{s} \mathbf{0})) : \mathcal{T}_{\text{Nat}}}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{}{\cdot \vdash \lambda^{\ell} x. ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}} \\
 \vdots \\
 \frac{}{\cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}}} \\
 \hline
 \cdot \vdash (\lambda^{\ell} x. ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \text{s } y) ; y) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash \lambda^{\ell} x. ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \text{s } y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}} \\
 \begin{array}{ccc}
 \vdots & & \vdots \\
 \vdots & & \vdots \\
 \vdots & & \vdots
 \end{array}
 \end{array}
 \frac{}{\cdot \vdash \text{s } (\text{s } \mathbf{0}) : \mathcal{T}_{\text{Nat}}}$$

$$\cdot \vdash (\lambda^{\ell} x. ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \text{s } y) ; y)) (\text{s } (\text{s } \mathbf{0})) : \mathcal{T}_{\text{Nat}}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{}{x : \mathcal{T}_{\text{Nat}} \vdash \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}}} \\
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash \lambda^\ell x. ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}} \\
 \begin{array}{ccc}
 \vdots & & \vdots \\
 \vdots & & \vdots \\
 \vdots & & \vdots
 \end{array}
 \end{array}$$

$$\frac{}{\cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}}}$$

$$\frac{}{\cdot \vdash (\lambda^\ell x. ((x \dot{=} \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y); y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \vdots \\
 \hline
 x : \mathcal{T}_{\text{Nat}}, y : \mathcal{T}_{\text{Nat}} \vdash \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\lambda^\ell x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash (x \doteq \mathbf{0}); \mathbf{none} : \mathcal{T}_{\text{Nat}}}{\vdots} \qquad \qquad \qquad \frac{\vdots}{x : \mathcal{T}_{\text{Nat}}, y : \mathcal{T}_{\text{Nat}} \vdash \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}}} \\
 \frac{\vdots}{x : \mathcal{T}_{\text{Nat}} \vdash \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}}} \\
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}} \\
 \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots \\
 \qquad \qquad \qquad \vdots \qquad \qquad \qquad \frac{\vdots}{\cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}}} \\
 \frac{\cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) \ (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}); \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y); y) \ (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}}
 \end{array}$$

Example of a Typing Derivation

$$\frac{}{x : \mathcal{T}_{\text{Nat}} \vdash x \doteq \mathbf{0} : \mathcal{T}_{\text{ok}}}$$

$$\frac{}{x : \mathcal{T}_{\text{Nat}} \vdash (x \doteq \mathbf{0}) ; \text{none} : \mathcal{T}_{\text{Nat}}}$$

$$\vdots$$

$$\frac{}{x : \mathcal{T}_{\text{Nat}}, y : \mathcal{T}_{\text{Nat}} \vdash \text{just } ((x \doteq \mathbf{s } y) ; y) : \mathcal{T}_{\text{Nat}}}$$

$$\frac{}{x : \mathcal{T}_{\text{Nat}} \vdash \nu y. \text{just } ((x \doteq \mathbf{s } y) ; y) : \mathcal{T}_{\text{Nat}}}$$

$$\frac{}{x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \mathbf{s } y) ; y) : \mathcal{T}_{\text{Nat}}}$$

$$\frac{}{\cdot \vdash \lambda^{\ell} x. ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \mathbf{s } y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}}$$

$$\vdots$$

$$\vdots$$

$$\frac{}{\cdot \vdash \mathbf{s } (\mathbf{s } \mathbf{0}) : \mathcal{T}_{\text{Nat}}}$$

$$\frac{}{\cdot \vdash (\lambda^{\ell} x. ((x \doteq \mathbf{0}) ; \text{none}) \oplus \nu y. \text{just } ((x \doteq \mathbf{s } y) ; y)) (\mathbf{s } (\mathbf{s } \mathbf{0})) : \mathcal{T}_{\text{Nat}}}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{}{x : \mathcal{T}_{\text{Nat}} \vdash x : \mathcal{T}_{\text{Nat}}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash x \doteq \mathbf{0} : \mathcal{T}_{\text{ok}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash (x \doteq \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \hline
 x : \mathcal{T}_{\text{Nat}}, y : \mathcal{T}_{\text{Nat}} \vdash \mathbf{just} ((x \doteq \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\lambda^\ell x. ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} \ y) ; y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash x : \mathcal{T}_{\text{Nat}} \quad x : \mathcal{T}_{\text{Nat}} \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}}}{x : \mathcal{T}_{\text{Nat}} \vdash x \dot{=} \mathbf{0} : \mathcal{T}_{\text{ok}}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash (x \dot{=} \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \frac{x : \mathcal{T}_{\text{Nat}} \vdash ((x \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash \lambda^\ell x. ((x \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \frac{\cdot \vdash \lambda^\ell x. ((x \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}} \quad \cdot \vdash \mathbf{s} (\mathbf{s} \ \mathbf{0}) : \mathcal{T}_{\text{Nat}}}{\cdot \vdash (\lambda^\ell x. ((x \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \dot{=} \mathbf{s} \ y) ; y)) (\mathbf{s} (\mathbf{s} \ \mathbf{0})) : \mathcal{T}_{\text{Nat}}}
 \end{array}$$

Example of a Typing Derivation

$$\begin{array}{c}
 \frac{}{x : \mathcal{T}_{\text{Nat}} \vdash x : \mathcal{T}_{\text{Nat}}} \quad \frac{}{x : \mathcal{T}_{\text{Nat}} \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash x \doteq \mathbf{0} : \mathcal{T}_{\text{ok}} \quad x : \mathcal{T}_{\text{Nat}} \vdash \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash (x \doteq \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \hline
 x : \mathcal{T}_{\text{Nat}} \vdash ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \lambda^\ell x. ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}} \rightarrow \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\lambda^\ell x. ((x \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((x \doteq \mathbf{s} y) ; y)) (\mathbf{s} (\mathbf{s} \mathbf{0})) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Subject Reduction

Theorem (Subject Reduction)

*Let P and Q be two programs such that $P \rightarrow Q$ and $\Gamma \vdash P : A$.
Then $\Gamma \vdash Q : A$.*

Proof.

The proof follows by case analysis of $P \rightarrow Q$.

Subject Reduction

Theorem (Subject Reduction)

*Let P and Q be two programs such that $P \rightarrow Q$ and $\Gamma \vdash P : A$.
Then $\Gamma \vdash Q : A$.*

Proof.

The proof follows by case analysis of $P \rightarrow Q$.

The interesting case is when $P = P_1 \oplus W \langle V \doteq W \rangle \oplus P_2 \rightarrow P_1 \oplus W \langle \mathbf{ok} \rangle^\sigma \oplus P_2$:
the unification algorithm preserves typing.



Type Preservation Example

$$\cdot \vdash ((s \ (s \ 0) \dot{=} 0) ; \mathbf{none}) \oplus \nu y. \mathbf{just} \ ((s \ (s \ 0) \dot{=} s \ y) ; y) : \mathcal{T}_{\text{Nat}}$$

Type Preservation Example

$$\cdot \vdash (\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}}$$
$$\vdots$$

$$\cdot \vdash ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}}$$

Type Preservation Example

$$\frac{\frac{\cdot \vdash \mathbf{s} \ (\mathbf{s} \ 0) \doteq 0 : \mathcal{T}_{\text{ok}}}{\cdot \vdash (\mathbf{s} \ (\mathbf{s} \ 0) \doteq 0) ; \mathbf{none} : \mathcal{T}_{\text{Nat}}}}{\vdots}$$
$$\frac{}{\cdot \vdash ((\mathbf{s} \ (\mathbf{s} \ 0) \doteq 0) ; \mathbf{none}) \oplus \nu y. \mathbf{just} \ ((\mathbf{s} \ (\mathbf{s} \ 0) \doteq \mathbf{s} \ y) ; y) : \mathcal{T}_{\text{Nat}}}$$

Type Preservation Example

$$\begin{array}{c}
 \vdots \\
 \hline
 \cdot \vdash s(s\ 0) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash s(s\ 0) \doteq 0 : \mathcal{T}_{\text{ok}} \\
 \hline
 \cdot \vdash (s(s\ 0) \doteq 0) ; \text{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \hline
 \cdot \vdash ((s(s\ 0) \doteq 0) ; \text{none}) \oplus \nu y. \text{just}((s(s\ 0) \doteq s\ y) ; y) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Type Preservation Example

$$\begin{array}{c}
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}} \quad \cdot \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0} : \mathcal{T}_{\text{ok}} \\
 \hline
 \cdot \vdash (\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \hline
 \cdot \vdash ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Type Preservation Example

$$\begin{array}{c}
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}} \quad \cdot \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0} : \mathcal{T}_{\text{ok}} \quad \cdot \vdash \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \hline
 \cdot \vdash ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Type Preservation Example

$$\begin{array}{c}
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}} \quad \cdot \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0} : \mathcal{T}_{\text{ok}} \quad \cdot \vdash \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \vdots \\
 \vdots \\
 \hline
 \cdot \vdash \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \dot{=} \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Type Preservation Example

$$\begin{array}{c}
 \vdots \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) : \mathcal{T}_{\text{Nat}} \quad \cdot \vdash \mathbf{0} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0} : \mathcal{T}_{\text{ok}} \quad \cdot \vdash \mathbf{none} : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash (\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0}) ; \mathbf{none} : \mathcal{T}_{\text{Nat}} \quad \vdots \\
 \vdots \\
 \vdots \\
 \hline
 y : \mathcal{T}_{\text{Nat}} \vdash \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}} \\
 \hline
 \cdot \vdash ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{0}) ; \mathbf{none}) \oplus \nu y. \mathbf{just} ((\mathbf{s} (\mathbf{s} \mathbf{0}) \doteq \mathbf{s} y) ; y) : \mathcal{T}_{\text{Nat}}
 \end{array}$$

Denotational Semantics for the λ^U -Calculus

Each base type α gets assigned a non-empty set S_α .

Interpretation of types ($\llbracket A \rrbracket$):

$$\begin{aligned}\llbracket \alpha \rrbracket &:= S_\alpha \\ \llbracket A \rightarrow B \rrbracket &:= \mathcal{P}(\llbracket B \rrbracket)^{\llbracket A \rrbracket}\end{aligned}$$

Denotational Semantics for the λ^U -Calculus

Each base type α gets assigned a non-empty set S_α .

Interpretation of types ($\llbracket A \rrbracket$):

$$\begin{aligned}\llbracket \alpha \rrbracket &:= S_\alpha \\ \llbracket A \rightarrow B \rrbracket &:= \mathcal{P}(\llbracket B \rrbracket)^{\llbracket A \rrbracket}\end{aligned}$$

Examples:

- $\llbracket \text{Bool} \rrbracket := \{\text{True}, \text{False}\}$
- $\llbracket \text{Nat} \rightarrow \text{Bool} \rrbracket := \mathcal{P}(\{\text{True}, \text{False}\})^{\mathbb{N}}$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket \mathbf{c} \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\}$$

$$\text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket \mathbf{c} \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket \mathbf{c} \rrbracket_\rho := \{R_{\mathbf{c}}\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

$$\llbracket M \doteq N \rrbracket_\rho := \{R_{ok} \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket N \rrbracket_\rho, a = b\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

$$\llbracket M \doteq N \rrbracket_\rho := \{R_{ok} \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket N \rrbracket_\rho, a = b\}$$

$$\llbracket M ; N \rrbracket_\rho := \{a \mid b \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

$$\llbracket M \doteq N \rrbracket_\rho := \{R_{ok} \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket N \rrbracket_\rho, a = b\}$$

$$\llbracket M ; N \rrbracket_\rho := \{a \mid b \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho\}$$

$$\llbracket \nu x^A. M \rrbracket_\rho := \{b \mid a \in \llbracket A \rrbracket, b \in \llbracket M \rrbracket_{\rho[x \mapsto a]}\}$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

$$\llbracket M \doteq N \rrbracket_\rho := \{R_{ok} \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket N \rrbracket_\rho, a = b\}$$

$$\llbracket M ; N \rrbracket_\rho := \{a \mid b \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho\}$$

$$\llbracket \nu x^A. M \rrbracket_\rho := \{b \mid a \in \llbracket A \rrbracket, b \in \llbracket M \rrbracket_{\rho[x \mapsto a]}\}$$

$$\llbracket \text{fail}^A \rrbracket_\rho := \emptyset$$

Denotational Semantics for the λ^U -Calculus

- Each variable x^A gets assigned an object $a \in \llbracket A \rrbracket$ via ρ :

$$\rho[x^A \mapsto a](y) := \begin{cases} a & \text{if } x = y \\ \rho(y) & \text{otherwise} \end{cases}$$

- Each term or program of type A is interpreted as a subset of $\llbracket A \rrbracket$.

Interpretation of terms and programs:

$$\llbracket x^A \rrbracket_\rho := \{\rho(x)\}$$

$$\llbracket c \rrbracket_\rho := \{R_c\}$$

$$\llbracket \lambda x^A. P \rrbracket_\rho := \{f\} =: \llbracket \lambda^\ell x^A. P \rrbracket_\rho \quad \text{with } f : \llbracket A \rrbracket \rightarrow \mathcal{P}(\llbracket B \rrbracket), \quad f(a) := \llbracket P \rrbracket_{\rho[x \mapsto a]}$$

$$\llbracket M N \rrbracket_\rho := \{b \mid f \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho, b \in f(a)\}$$

$$\llbracket M \doteq N \rrbracket_\rho := \{R_{\text{ok}} \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket N \rrbracket_\rho, a = b\}$$

$$\llbracket M ; N \rrbracket_\rho := \{a \mid b \in \llbracket M \rrbracket_\rho, a \in \llbracket N \rrbracket_\rho\}$$

$$\llbracket \nu x^A. M \rrbracket_\rho := \{b \mid a \in \llbracket A \rrbracket, b \in \llbracket M \rrbracket_{\rho[x \mapsto a]}\}$$

$$\llbracket \text{fail}^A \rrbracket_\rho := \emptyset$$

$$\llbracket M \oplus P \rrbracket_\rho := \llbracket M \rrbracket_\rho \cup \llbracket P \rrbracket_\rho$$

Denotational Semantics - Example

Let us consider $\llbracket T \rrbracket := \{\text{True}\}$, $\llbracket F \rrbracket := \{\text{False}\}$, and:

$$\mathbf{or} := \lambda^{\ell}x.\lambda^{\ell'}y.(\textcolor{brown}{x \doteq T; T}) \oplus (\textcolor{blue}{y \doteq T; T}) \oplus (\textcolor{gray}{x \doteq F; ((y \doteq F); F)})$$

Denotational Semantics - Example

Let us consider $\llbracket T \rrbracket := \{\text{True}\}$, $\llbracket F \rrbracket := \{\text{False}\}$, and:

$$\mathbf{or} := \lambda^{\ell}x.\lambda^{\ell'}y.(\textcolor{brown}{x \doteq T; T}) \oplus (\textcolor{blue}{y \doteq T; T}) \oplus (\textcolor{gray}{x \doteq F; ((y \doteq F); F)})$$

Then:

$$\llbracket \mathbf{or} \ T \ F \rrbracket$$

Denotational Semantics - Example

Let us consider $\llbracket T \rrbracket := \{\text{True}\}$, $\llbracket F \rrbracket := \{\text{False}\}$, and:

$$\text{or} := \lambda^{\ell}x.\lambda^{\ell'}y.(\boxed{x \doteq T; T}) \oplus (\boxed{y \doteq T; T}) \oplus (\boxed{x \doteq F; ((y \doteq F); F)})$$

Then:

$$\begin{aligned} & \llbracket \text{or } T \ F \rrbracket \\ = & \llbracket (\boxed{x \doteq T; T}) \oplus (\boxed{y \doteq T; T}) \oplus (\boxed{x \doteq F; ((y \doteq F); F)}) \rrbracket_{[x \mapsto \text{True}][y \mapsto \text{False}]} \end{aligned}$$

Denotational Semantics - Example

Let us consider $\llbracket T \rrbracket := \{\text{True}\}$, $\llbracket F \rrbracket := \{\text{False}\}$, and:

$$\text{or} := \lambda^{\ell}x.\lambda^{\ell'}y.(\boxed{x \doteq T; T}) \oplus (\boxed{y \doteq T; T}) \oplus (\boxed{x \doteq F; ((y \doteq F); F)})$$

Then:

$$\begin{aligned} & \llbracket \text{or } T \ F \rrbracket \\ = & \llbracket (\boxed{x \doteq T; T}) \oplus (\boxed{y \doteq T; T}) \oplus (\boxed{x \doteq F; ((y \doteq F); F)}) \rrbracket_{[x \mapsto \text{True}][y \mapsto \text{False}]} \\ = & \dots \end{aligned}$$

Denotational Semantics - Example

Let us consider $\llbracket T \rrbracket := \{\text{True}\}$, $\llbracket F \rrbracket := \{\text{False}\}$, and:

$$\text{or} := \lambda^{\ell}x.\lambda^{\ell'}y.(x \doteq T; T) \oplus (y \doteq T; T) \oplus (x \doteq F; ((y \doteq F); F))$$

Then:

$$\begin{aligned} & \llbracket \text{or } T \ F \rrbracket \\ = & \llbracket (x \doteq T; T) \oplus (y \doteq T; T) \oplus (x \doteq F; ((y \doteq F); F)) \rrbracket_{[x \mapsto \text{True}][y \mapsto \text{False}]} \\ = & \dots \\ = & \{\text{True}\} \end{aligned}$$

Denotational Semantics - Irrelevance and Compositionality

Irrelevance

If $\Gamma \vdash X : A$ and ρ, ρ' coincide on $\text{fv}(X)$ then $\llbracket X \rrbracket_\rho = \llbracket X \rrbracket_{\rho'}$.

Compositionality

- 1 $\llbracket P \oplus Q \rrbracket = \llbracket P \rrbracket \cup \llbracket Q \rrbracket$
- 2 $\llbracket W \langle M \rangle \rrbracket_\rho = \{b \mid a \in \llbracket M \rrbracket_\rho, b \in \llbracket W \rrbracket_{\rho[\square \mapsto a]}\}$

(Weak) Soundness

Theorem

If $\Gamma \vdash P : A$ and $P \rightarrow Q$ then $\llbracket P \rrbracket \supseteq \llbracket Q \rrbracket$.

Proof.

By case analysis of $P \rightarrow Q$.

It is worth noting that in all the cases but the *fail*¹ rule, it holds that $\llbracket P \rrbracket = \llbracket Q \rrbracket$. □

¹ $W\langle V \doteq W \rangle \rightarrow W\langle \text{fail} \rangle$ if $\text{mgu}(\{V \doteq W\})$ fails.

(Weak) Soundness

Theorem

If $\Gamma \vdash P : A$ and $P \rightarrow Q$ then $\llbracket P \rrbracket \supseteq \llbracket Q \rrbracket$.

Proof.

By case analysis of $P \rightarrow Q$.

It is worth noting that in all the cases but the $fail^1$ rule, it holds that $\llbracket P \rrbracket = \llbracket Q \rrbracket$. □

Example:

$$(\lambda^{\ell_1} x. x) \doteq \lambda^{\ell_2} x. x \rightarrow fail^A$$

¹ $W\langle V \doteq W \rangle \rightarrow W\langle fail \rangle$ if $mgu(\{V \doteq W\})$ fails.

(Weak) Soundness

Theorem

If $\Gamma \vdash P : A$ and $P \rightarrow Q$ then $\llbracket P \rrbracket \supseteq \llbracket Q \rrbracket$.

Proof.

By case analysis of $P \rightarrow Q$.

It is worth noting that in all the cases but the *fail*¹ rule, it holds that $\llbracket P \rrbracket = \llbracket Q \rrbracket$. □

Example:

$$(\lambda^{\ell_1} x. x) \doteq \lambda^{\ell_2} x. x \rightarrow \text{fail}^A$$

But:

- $\llbracket \lambda^{\ell_1} x. x \doteq \lambda^{\ell_2} x. x \rrbracket = \{\text{R}_{\text{ok}}\}$
- $\llbracket \text{fail} \rrbracket = \emptyset$

¹ $W \langle V \doteq W \rangle \rightarrow W \langle \text{fail} \rangle$ if $\text{mgu}(\{V \doteq W\})$ fails.

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^ Q$.*

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^ Q$.*

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:

$$\lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \qquad \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P$$

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^* Q$.

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:

$\lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P$

$\lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P$

↓
ok

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^* Q$.

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:

$\lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P$



ok

$\lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P$



fail

Completeness

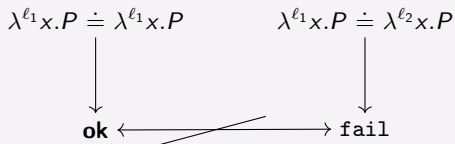
Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^* Q$.

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:



Completeness

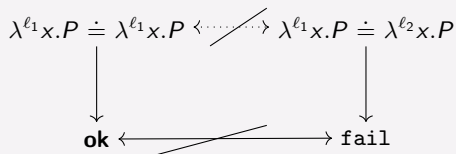
Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^* Q$.

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:



Completeness

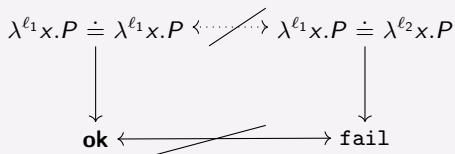
Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^* Q$.

Examples Showing That Completeness Does Not Hold

- The OS is **location sensitive**, DS is not:

$\llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_1} x. P \rrbracket = \llbracket \lambda^{\ell_1} x. P \doteq \lambda^{\ell_2} x. P \rrbracket$, but:



- The OS is sensitive to **stuck terms**, DS is not:

$\llbracket (x \ y) ; \mathbf{ok} \rrbracket = \llbracket \mathbf{ok} \rrbracket$, but:

$(x \ y) ; \mathbf{ok} \not\leftrightarrow^* \mathbf{ok}$

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^ Q$.*

Examples Showing That Completeness Does Not Hold

- The OS allows for **multiple results**, DS does not:

$\llbracket \mathbf{c} \oplus \mathbf{c} \rrbracket = \llbracket \mathbf{c} \rrbracket$, but:

$$\mathbf{c} \oplus \mathbf{c} \not\leftrightarrow^* \mathbf{c}$$

Completeness

Theorem

If $\Gamma \vdash P : A$ and $\llbracket P \rrbracket = \llbracket Q \rrbracket$ then $P \leftrightarrow^ Q$.*

Examples Showing That Completeness Does Not Hold

- The OS allows for **multiple results**, DS does not:

$\llbracket \mathbf{c} \oplus \mathbf{c} \rrbracket = \llbracket \mathbf{c} \rrbracket$, but:

$$\mathbf{c} \oplus \mathbf{c} \leftrightarrow^* \mathbf{c}$$

- The OS is **not extensional**, DS it is:

$\llbracket \mathbf{c} \rrbracket = \llbracket \lambda x. \mathbf{c} \ x \rrbracket$, but:

$$\mathbf{c} \leftrightarrow^* \lambda x. \mathbf{c} \ x$$

We explored two alternatives:

Denotational Semantics With Memory

- The DS incorporates a notion of state (through the memory).
- A λ -abstraction is interpreted as an operation allocating space in memory.
- An allocated λ -abstraction is a value pointing towards the memory position ℓ .
- **Problem:** Cyclic dependencies appear in the memory, thus it is not clear how to prove the semantics is compositional.

Alternative Denotational Semantics for λ^U

Allowing Closures in λ^U

- Instead of λ -abstractions, we consider terms of the form $(\lambda x.P)[x_1 \setminus V_1] \dots [x_n \setminus V_n]$ (there are no locations).
- Two closures unify iff they are *equal* modulo permutation of $[x_i \setminus V_i]$.
- This new variant of λ^U has been proven to be confluent too.
- **Problem:** Setting the interpretation domain for an arrow type is non-trivial: a closure may encapsulate data types bigger than its type.

Alternative Denotational Semantics for λ^U

Allowing Closures in λ^U

- Instead of λ -abstractions, we consider terms of the form $(\lambda x. P)[x_1 \setminus V_1] \dots [x_n \setminus V_n]$ (there are no locations).
- Two closures unify iff they are *equal* modulo permutation of $[x_i \setminus V_i]$.
- This new variant of λ^U has been proven to be confluent too.
- **Problem:** Setting the interpretation domain for an arrow type is non-trivial: a closure may encapsulate data types bigger than its type.

“To deal with this difficulty, we try to give a denotational semantics by interpreting the types as complete partial orders, rather than sets, and defining $\llbracket A \rightarrow B \rrbracket$ as a filtering limit. Defining the denotational semantics in this way turns out to be technically complex, and we leave its in-depth study as future work.”

- Revise the operational behaviour of λ^U .
- Revise the type system: dependent types?
- Provide a DS for which the OS is sound and complete.
- Relate this language with others such as Verse, or even pure functional languages.



Thanks for your
attention!